



Component Communication

Persistent Interactive | Persistent University



Key Learning Points

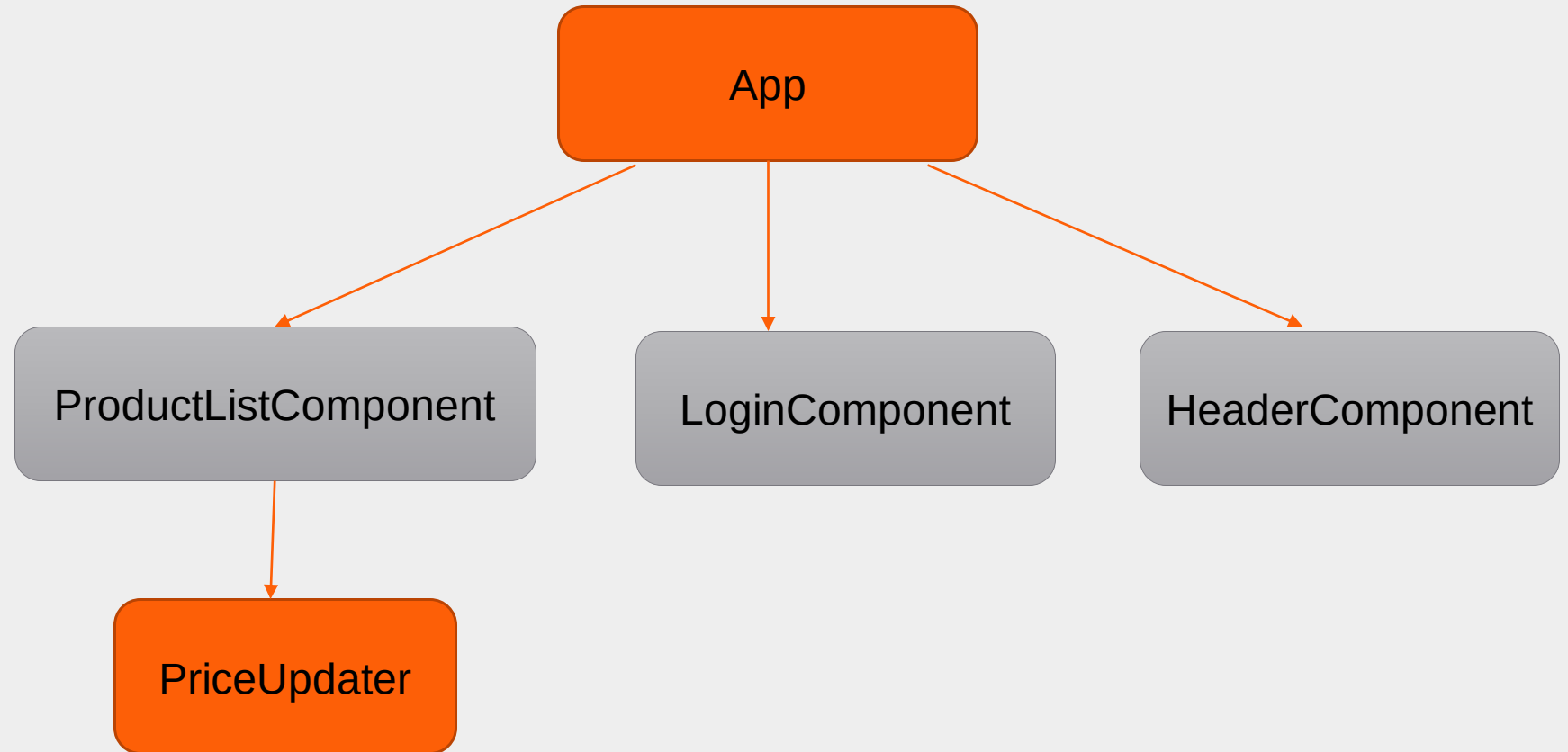
- Why the need for Components to communicate
- Three approaches to communication
- Concept of Observable
- Subscribing to Observables

Scenarios for Communication

- **LoginComponent** can share authentication status to **MenuComponent**.
- **LoginComponent** can share name of logged-in user to **HeaderComponent**.
- **ProfileComponent** can share updated username to **HeaderComponent**.
- **CartComponent** can share list of items purchased to **ProductsComponent**.
- **InventoryComponent** can share updated items to **ProductsComponent**.

Passing data between components

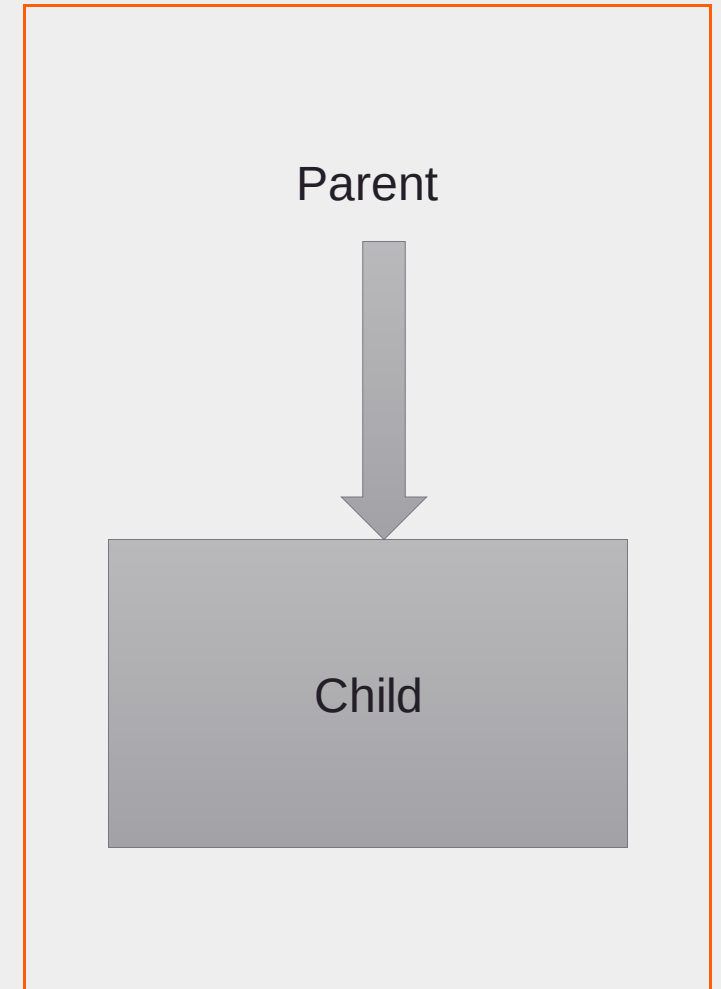
- Parent to Child
- Child to Parent
- Child to Child



Parent to Child

- **@Input()** annotation
- Mark the variable/object expected from parent

```
export class ChildComponent
  @Input()
  name:string;
  constructor() { }
```



Set the input property in the parent

- The property will be set by the parent.

```
@Component({  
  selector: 'my-app',  
  template: `<h1>Hello {{parentname}}</h1>  
              <child [name]="parentname"></child>  
            `,  
})
```

When property names do not match

- There may be times when the input property name is different in Child and Parent Components.
- We can specify the name in the @Input annotation.
- Suppose parent property is **'username'**

```
template: `<h1>Hello {{parentname}}</h1>
          <child [username]="parentname"></child>
          `
```

```
export class ChildComponent
  @Input('username')
  name:string;
```

Specify the parent property
name

Angular Ivy Validation for @Input()

- If a child component has a variable with @Input as below:
- `@Input() title: string;`
- Suppose this value is set in the parent component as below:
- `<child [title]="titlevalue"></child>`
- Lets say the titlevalue variable in the parent component is set to null as below:
- ```
export class ParentComponent {
 title = null;
}
```
- This will result in compilation error saying **null is not assignable to string.**
- Solution in next slide.

## Resolving the error for @Input with null value

1. Cast the type of variable to any.

- `<child [title]="$any(titlevalue)"></child>`

2. Define the title property in child as below:

```
@Input() title: string | null ;
```

## Accessing Child's properties and methods

- How can a Parent access properties and methods of child ?
- Use of **Template Variable**
- Define a template variable with #
- **It provides access to all properties and methods of the Child class**

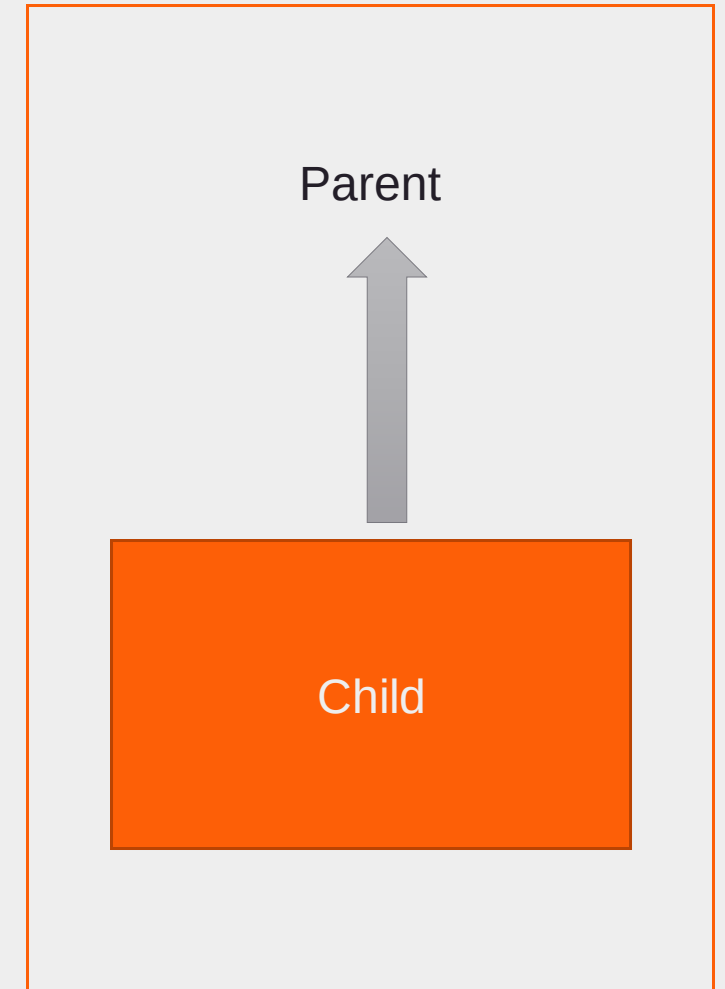
```
<child #childRef></child>
```

```
<div>Count value from Child2: {{childRef.getCount()}}</div>
```

## Child to Parent

- Child can share data or notify the parent via **Events**.
- **@Output()** annotation
- Need an instance of **EventEmitter**
- This instance needs to be marked with @Output
- Here, we create an event “**myevent**”

```
export class ChildComponent {
 @Output()
 myevent:EventEmitter<string> = new EventEmitter();
}
```



## Custom Events

- Note that these are **Custom events**
- **Data(any type) can be passed in Custom events**
- Custom events can be raised based on application logic.
  - Eg: After profile is updated, raise an **'updated'** event.
  - Eg: After login is successful raise an **'login'** event.

```
<button (click)="raiseMyEvent()">Send</button>
```

```
raiseMyEvent():void{
 this.myevent.emit('PROD102');
 console.log('raised event');
}
```

Raise the event **'myevent'** and  
also pass some data

## Parent has to handle the event

- Just like we handle (click) event, we handle the **'myevent'**

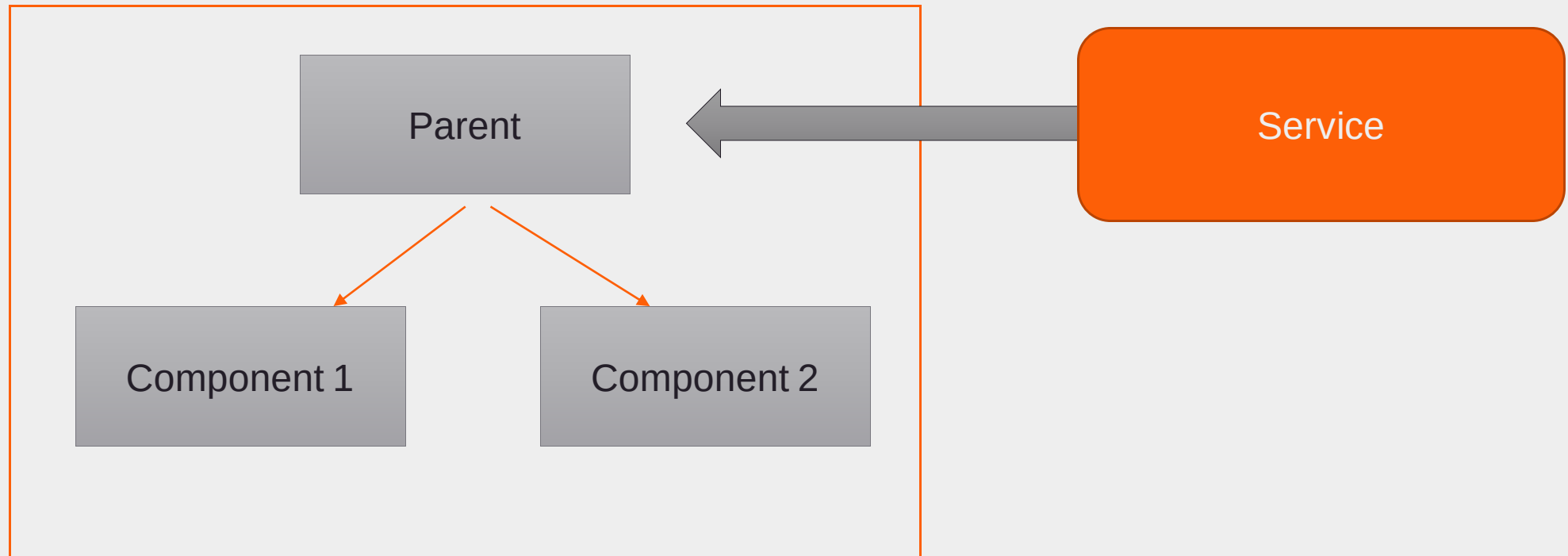
```
<child [username]="parentname" (myevent)="handleMyEvent($event)"></child>
```

```
|
handleMyEvent(arg:any){
 alert('Event raised by child with values: '+arg);
}
```

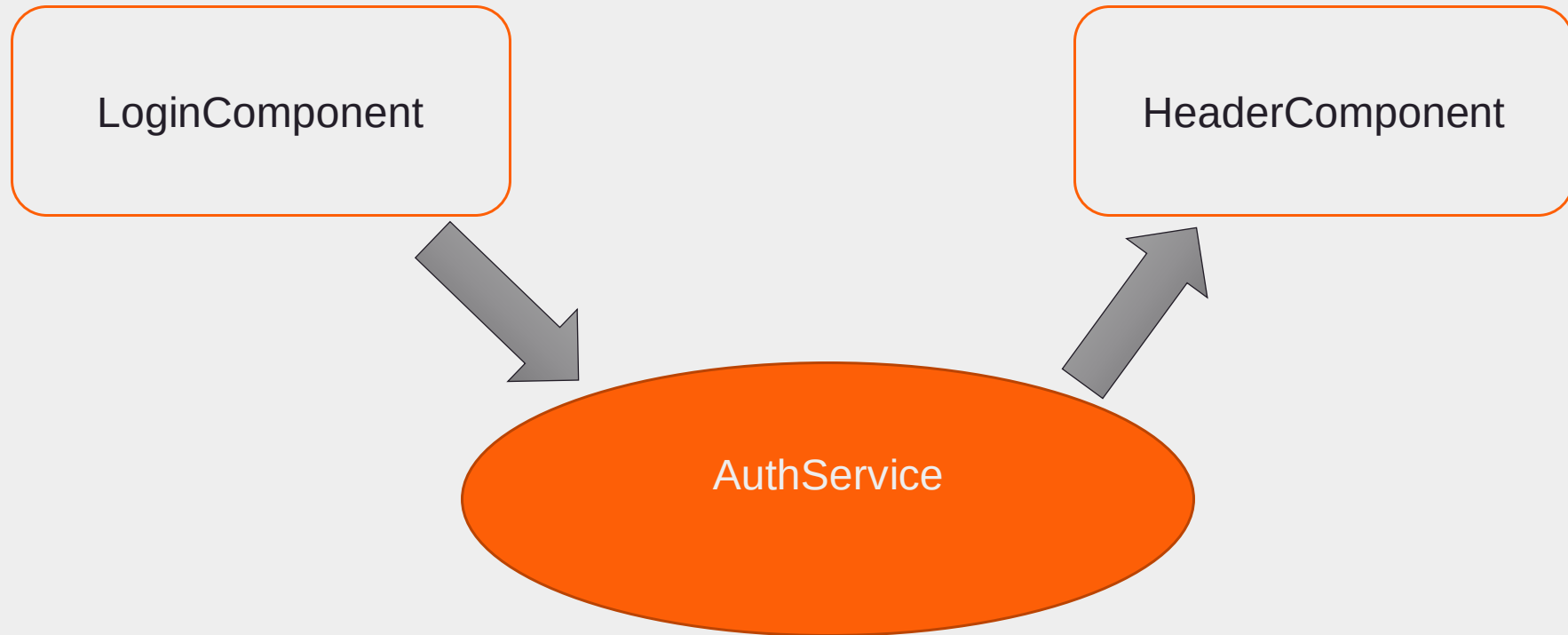
- Important point is to pass the keyword **\$event** to access the data passed by the child.

## Sharing data between sibling components

- Data between the child components is exchanged via a **Service**
- This communication is based on the Observable pattern.
- This internally is based on events.



## Communication between Sibling Components



## A new Async programming concept - The Stream

- Building programs with the concepts of Streams.
- Eg: Stream of numeric values,

3,4,5,6,7,8,9

- Eg: Stream of mouse click events,

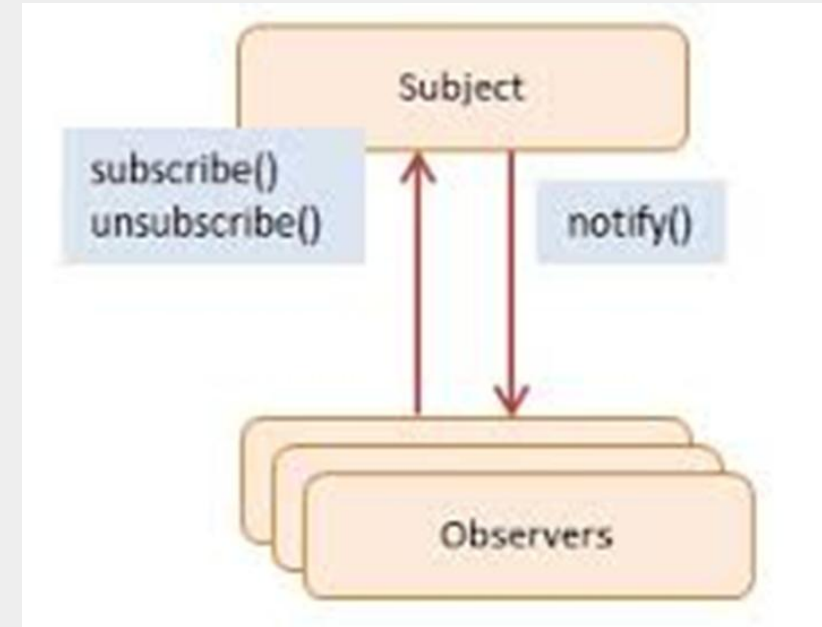
(10,200),(100,50),(430,280)

- Eg: Stream of server side data

[{name:'Mack','age':35}, {name:'Jack','age':45}, {name:'Jim','age':25}]

## A new ASync primitive : Observable

- To work with streams, we need:
  - Create streams
  - Subscribe to streams
  - Read data from streams
  - Combine/Transform streams.
- **Observable enables to work with streams.**
- The combination of a stream with a set of functional operators to transform streams leads us to the concept of Observable.
- Functional Reactive Programming

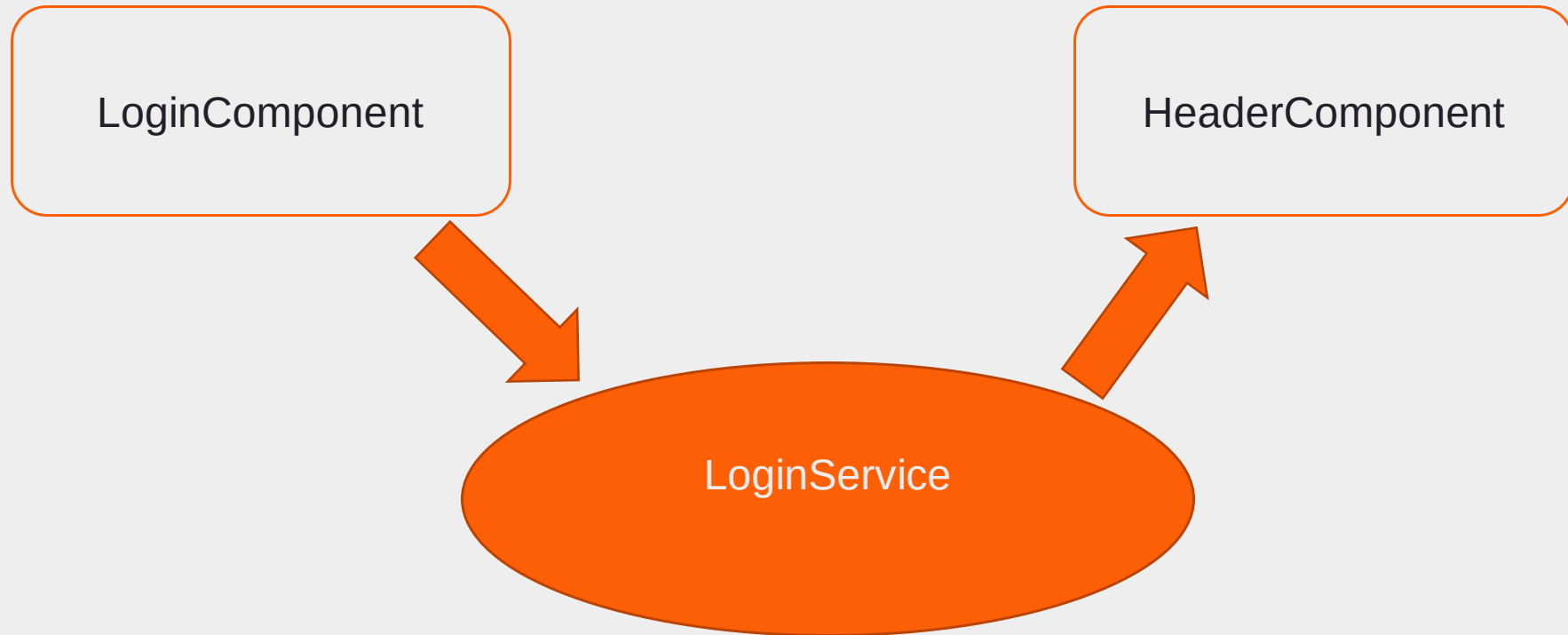


## RxJs Library

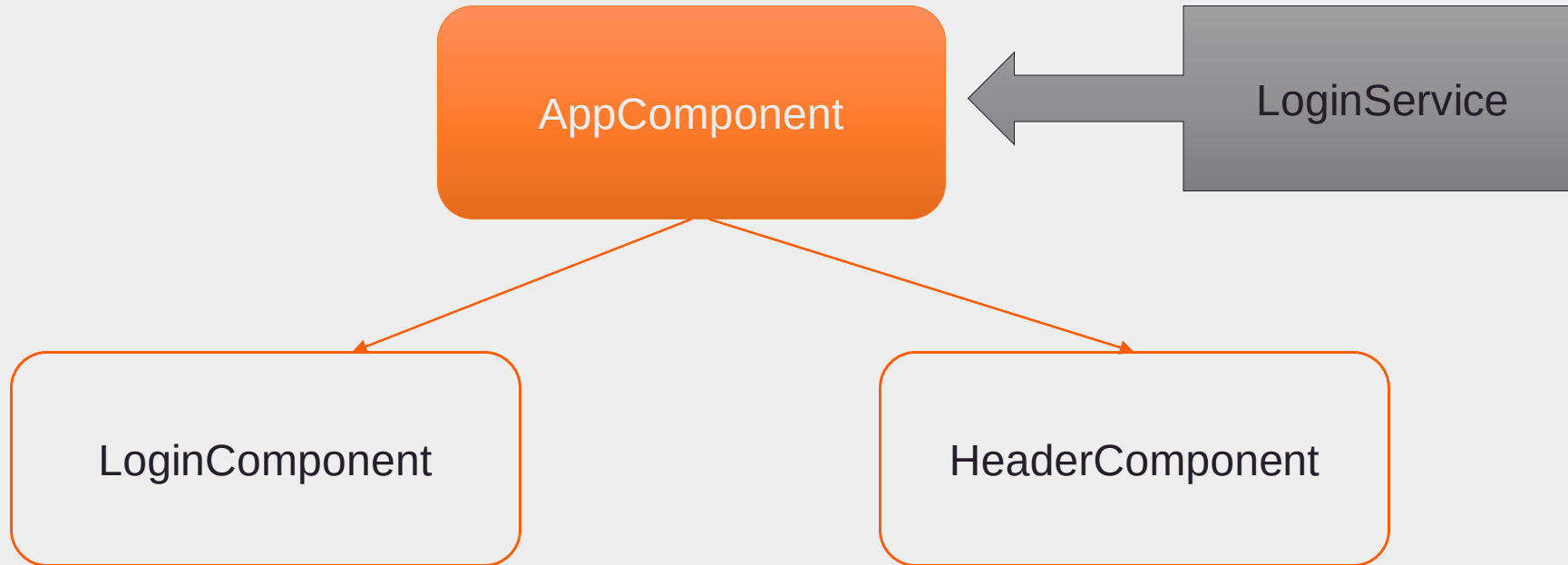
- RxJs is an implementation of Asynchronous Observables for Javascript
- RxJs stands for **Reactive Extensions for Javascript**.

```
{} package.json x
{} package.json
15 "@angular/common": "~9.1.4",
16 "@angular/compiler": "~9.1.4",
17 "@angular/core": "~9.1.4",
18 "@angular/forms": "~9.1.4",
19 "@angular/platform-browser": "~9.1.4",
20 "@angular/platform-browser-dynamic": "~9.1.4",
21 "@angular/router": "~9.1.4",
22 "rxjs": "~6.5.4",
23 "tslib": "^1.10.0",
24 "zone.js": "~0.10.2"
25 },
```

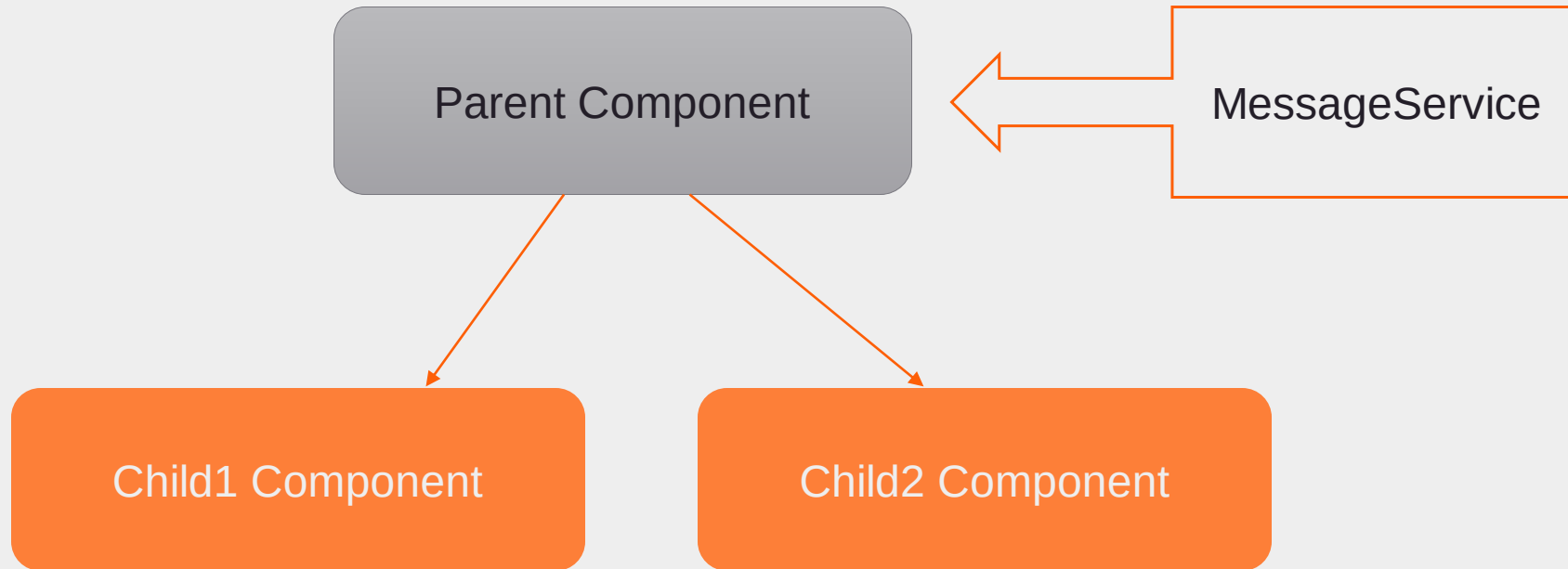
## Communication between Sibling Components



## Example



## Single instance of MessageService



- A single instance of the service is injected in the parent.
- The **same service instance** is available to all child components in the tree.

## Implementing the MessageService

- Import the classes from Rxjs

```
import {Subject} from 'rxjs/Subject';
import {Observable} from 'rxjs/Observable'
```

- Define the

```
@Injectable()
export class SharedService {
 private msg = new Subject<string>();
```

- Define an

```
public msgBroadcaster = this.msg.asObservable();
```

## Emit the Event

- Define a method which emits an event.
- It will emit an event on the subject '**msg**'

```
sendMessage(message:string){
 this.msg.next(message);
}
```

## Who will invoke the service?

- Lets say Component1 wants to send a message to Component 2.
- Component1 will invoke the service.

```
<button (click)="sendMessage(msg.value)">Send Message</button>
```

```
export class ChildComponent {
 sendMessage(message:string):void{
 this._sharedservice.sendMessage(message);
 }
}
```

## Who will receive the event?

- The Component who has subscribed to the changes on the subject, will receive the event.

- Display this

```
export class Child2Component {
 messageStr:string;
 constructor(private _sharedservice:SharedService) {
 this._sharedservice.msgBroadcaster.subscribe(message => {
 this.messageStr = message; });
 }
}
```

```
template: `<div>Child2 Component
 <div>Data from Child1 Component: {{messageStr}}</div>
 </div>`,
```

## Where to inject the service?

- Add to list of providers in the PARENT COMPONENT.
- **This ensures same instance of service is used by all the child components.**
- Add to list of providers
- Inject in constructor

```
@Component({
 selector: 'my-app',
 template: `<h1>Hello {{parentname}}

 <child [username]="parentname">
 <child2></child2>
 </child>
 `,
 providers: [SharedService]
})
```

```
constructor(private _sharedservice: SharedService){

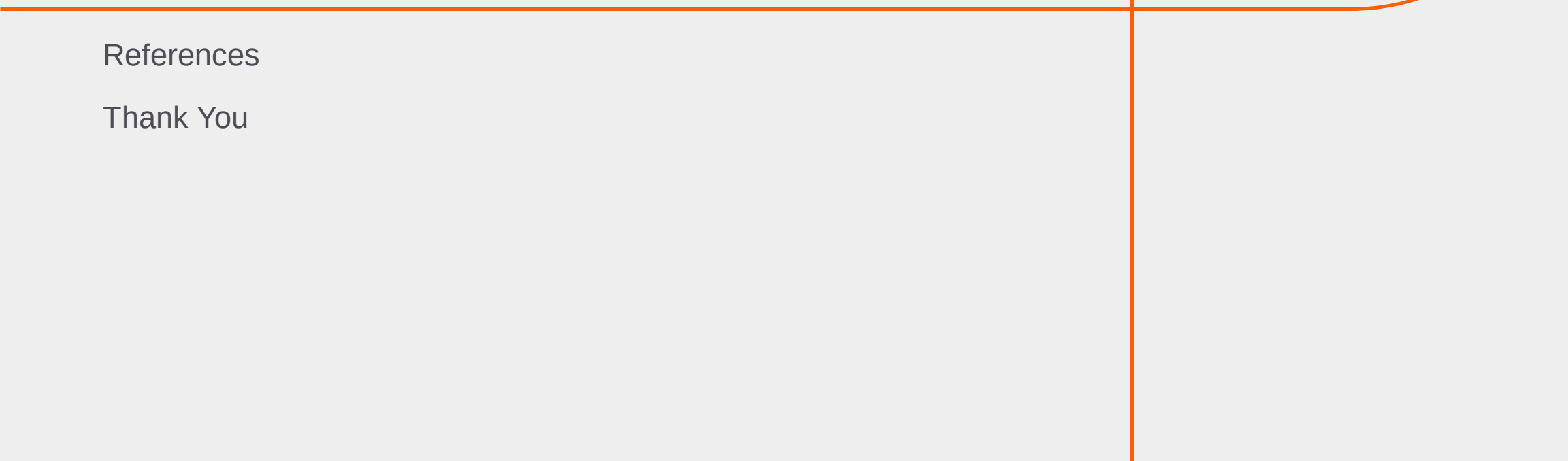
}
```

## Summary: Session #

With this we have come to an end of our session, where we discussed about ....

- Using the RxJs library
- Parent to Child communication
- Child to Parent communication
- Inter-component communication
- Concept of Observables

# Appendix



References

Thank You

## Reference Links

- Input and Output
  - <https://www.themarketingtechnologist.co/building-nested-components-in-angular-2/>
- RxJS and Observables
  - <http://blog.angular-university.io/functional-reactive-programming-for-angular-2-developers-rxjs-and-observables/>
- Communication between sibling components- Example
  - <http://jasonwatmore.com/post/2016/12/01/angular-2-communicating-between-components-with-observable-subject>
- Promises and Observables
  - <http://moduscreate.com/observables-and-promises/>

## Key Contacts :

### *Persistent University :*

- Archana Ghatigar  
[archana\\_ghatigar@persistent.co.in](mailto:archana_ghatigar@persistent.co.in)



# Thank you!

Persistent Interactive | Persistent University

